

Muhamed Ahmed Portfolio

Complete Project Documentation

Next.js 16

React 19

TypeScript

Prisma 5

PostgreSQL

Vercel Blob

Vercel

Live site: **<https://muhamed-ahmed-portfolio.vercel.app>**

Admin panel: **<https://muhamed-ahmed-portfolio.vercel.app/admin>**

September 2026

1. Project Overview

A personal portfolio website for Muhamed Ahmed Shifaw, Software Developer. Originally a static HTML site, it was rebuilt as a modern full-stack application with the same visual design plus a complete admin content-management system.

Key capabilities

- **Public portfolio** — Home, About, Experience, Projects, Skills, CV, and Contact pages, all rendered from live database content (no code changes or rebuilds needed for content updates).
- **Admin dashboard** (/admin) — password-protected panel where the owner edits every section of the site: About (including photo upload), Experience, Projects, Skills, CV/Resume (including PDF upload), Contact, and theme colors.
- **Database-backed content** — every text section lives in PostgreSQL. Public pages read the database on every request, so admin edits appear site-wide instantly.
- **File uploads via Vercel Blob** — profile photos and CV PDFs upload to Vercel's Blob CDN; the public URL is persisted to the database automatically within the same upload request.
- **Theme colors** — primary and background colors editable in admin, applied via CSS custom properties.
- **Dark/light mode** — with a pre-hydration script to avoid flash-of-wrong-theme and React hydration mismatch.

2. Technology Choices

Layer	Technology	Why
Framework	Next.js 16, App Router	Server rendering, co-located API routes, Vercel-native
UI	React 19 + plain CSS	Ported 1:1 from the original design; no CSS framework dependency
Language	TypeScript 5	Type safety across content shapes and API contracts
ORM	Prisma 5.22 (pinned)	Typed DB access; newer majors pull an oversized dependency tree that stalled installs
Database	PostgreSQL (Neon via Vercel Postgres integration)	Serverless-friendly, auto-injected env vars, pooled connections
File storage	Vercel Blob (@vercel/blob)	Zero-config CDN-backed uploads, OIDC auth in production
Hosting	Vercel	Same platform as DB and Blob; instant deploys and preview builds

3. Architecture

3.1 Request flows

Browser

- | Public pages → Next.js server components → lib/db getContent() → Postgres
- └ /admin → AdminDashboard (client component)
 - | text saves → PUT /api/content (auth-guarded) → Postgres
 - └ uploads → POST /api/upload-profile | /api/upload-cv
 - Vercel Blob → URL persisted to Postgres in the same request

3.2 Directory map

Path	Role
<code>app/layout.tsx</code>	Root layout; theme init script that runs before hydration
<code>app/(site)/...</code>	Public pages (home, about, experience, projects, skills, cv, contact) — all <code>force-dynamic</code> , all reading content via <code>lib/db</code>
<code>app/admin/</code>	Admin page + layout (renders AdminDashboard)
<code>app/api/content/route.ts</code>	GET public read · PUT auth-guarded save (patch semantics)
<code>app/api/auth/login logout</code>	Session cookie issue / clear
<code>app/api/auth/password</code>	PUT — change admin password (verifies current, updates the ADMIN_PASSWORD env var via the Vercel REST API)
<code>app/api/upload-profile/route.ts</code>	POST — photo → Vercel Blob → URL saved to the About row
<code>app/api/upload-cv/route.ts</code>	POST — PDF → Vercel Blob → URL saved to the Cv row
<code>components/AdminDashboard.tsx</code>	The entire admin UI (tabs per section, per-card Save)
<code>components/Sidebar.tsx</code>	Public sidebar with photo, navigation, theme toggle
<code>lib/content.ts</code>	TypeScript types + default content (client-safe)
<code>lib/db.ts</code>	Prisma client (singleton), <code>getContent()/saveContent()</code> , Vercel env fallback chain, DB-down circuit breaker
<code>lib/auth.ts</code>	HMAC-signed session cookie, 7-day TTL, timing-safe verification
<code>lib/cv-utils.ts</code>	Client-safe CV merge helpers
<code>lib/vercel-env.ts</code>	Vercel REST API helper for the password-change flow
<code>prisma/schema.prisma</code>	10 models (see section 4)
<code>prisma/migrations/0_init/</code>	Initial SQL migration
<code>prisma/seed.mjs</code>	Loads <code>data/content.json</code> into Postgres
<code>data/content.json</code>	Seed source + canonical content shape
<code>public/</code>	Static assets (original photo/CV still bundled)
<code>next.config.ts</code>	<code>images.remotePatterns</code> allows <code>*.public.blob.vercel-storage.com</code> so <code>next/image</code> can optimize Blob photos

4. Database Schema (Prisma models)

Single-row sections (fixed id = 1)

Model	Fields
About	name, title, bio, location, profilePhoto, updatedAt
Contact	email, phone, github, linkedin, twitter, telegram, updatedAt
ColorTheme	primary, bg, updatedAt
Cv	pdfUrl, profile, updatedAt; has many CvStat / CvEducation / CvLanguage

List sections (ordered; the order field is unique)

Model	Fields
Project	title, desc, role, tech (string array)
Experience	title, company, location, date, responsibilities (string array)
Skill	name
CvStat	label, number (belongs to Cv)
CvEducation	degree, institution, date (belongs to Cv)
CvLanguage	language, level (belongs to Cv)

`lib/db.ts` assembles these rows into the single `Content` object the UI consumes (`getContent()`); `saveContent(partial)` patches only the sections present in the request — list sections sync atomically inside a transaction (delete-missing, upsert-rest). If the database is unreachable, a 30-second circuit breaker serves seeded defaults instead of erroring, keeping the public site online.

5. Admin Panel Guide

Log in at `/admin` with the `ADMIN_PASSWORD` value. Sections (left sidebar tabs):

1. **About Me** — name, title, bio, location; Profile Photo widget (Choose File → Upload & Save; uploads to Blob and updates the site immediately; persisting a `blob`: preview URL is blocked by a guard).
2. **Experience** — job list with add/remove, per-job responsibilities; Save Experience.
3. **Projects** — add/remove/edit projects (title, description, role, tech list); Save Projects.
4. **Skills** — comma-separated skill list; Save Skills.
5. **CV / Resume** — PDF upload (same Blob flow), profile bio, stats, education, languages; Save CV.
6. **Contact** — email, phone, GitHub, LinkedIn, Twitter/X, Telegram; Save Contact.
7. **Colors** — primary and background color pickers; Apply Colors.
8. **Account & Security** — change password (verifies the current password; minimum 8 characters). The new value is stored in the Vercel project's `ADMIN_PASSWORD` env var via the Vercel REST API; it takes

effect on the next deployment because serverless functions snapshot env vars at deploy time. All saves go through `PUT /api/content` (or the upload routes), which requires a valid admin session cookie.

6. Authentication & Security

- **Session:** after login, an HMAC-SHA256–signed token (payload = expiry, 7-day TTL) is set as an `httpOnly`, `sameSite=lax`, `secure` -in-production cookie named `admin_session`. Verification is timing-safe.
- **Secrets:** `AUTH_SECRET` signs tokens; `ADMIN_PASSWORD` gates login. Production values live in Vercel project settings, never in the repo (`.env*` is git-ignored).
- **Dev defaults:** insecure fallbacks (`admin123`, dev secret) warn loudly but exist only outside production.
- **Password change:** authenticated-only; verifies the current password before accepting a new one.
- **Public API:** `GET /api/content` exposes read-only public data; all writes are session-guarded.

Known hardening opportunities (not yet implemented): login rate limiting; moving the password into the database as a bcrypt hash so changes take effect instantly.

7. Environment Variables

Variable	Purpose	Set where
<code>DATABASE_URL</code>	Postgres connection (pooled; Prisma adds <code>pgbouncer=true&connection_limit=1</code> automatically)	Vercel auto-injects (Neon integration); <code>.env.local</code> locally
<code>POSTGRES_PRISMA_URL</code> / <code>POSTGRES_URL</code>	Fallback chain used by lib/db.ts if <code>DATABASE_URL</code> is absent	Auto-injected by Vercel
<code>ADMIN_PASSWORD</code>	Admin login password	Vercel project env vars
<code>AUTH_SECRET</code>	Session-cookie signing secret	Vercel project env vars
<code>BLOB_STORE_ID</code> + runtime OIDC token	Blob auth in production (no static token needed)	Auto-injected by the Blob integration
<code>BLOB_READ_WRITE_TOKEN</code>	Blob auth alternative for local dev / CLI	Optional
<code>VERCEL_TOKEN</code>	Enables in-app password changes via the Vercel REST API	Hidden project env var

Generate `AUTH_SECRET`: `node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`

8. Production Deployment (current state)

- **Project:** `muhamed-portfolio`, linked via `.vercel/project.json`.
- **Database:** Neon Postgres connected through Vercel's Storage integration; `DATABASE_URL` and `POSTGRES_*` vars injected automatically for all environments. Initial migration deployed; content seeded from `data/content.json`.
- **Blob store:** `portfolio-blob` (`store_Edvho7f7UGv0Sa3X`, public access, region iad1), OIDC-connected. Verified end-to-end in production: photo and PDF uploads return Blob URLs that are publicly served and persisted to the database.
- **Domain:** clean production alias **`https://muhamed-ahmed-portfolio.vercel.app`**. Vercel auto-appends the account name to raw deployment URLs; the alias hides it. Deployment protection is disabled because the admin area has its own authentication.
- **Images:** `next.config.ts` allows `*.public.blob.vercel-storage.com` in `images.remotePatterns` — without this, Next's image optimizer returns 400 for Blob photos and the UI falls back to initials.
- **Build:** `prisma generate && next build`; all DB-reading pages are `force-dynamic`, so builds never touch the database.

Redeploying: `npx vercel --prod` (or push to the connected Git repo). After changing the admin password through the panel, redeploy for it to take effect.

9. Local Development

```
npm install
cp .env.example .env.local      # fill in DATABASE_URL, ADMIN_PASSWORD, AUTH_SECRET
npx prisma migrate deploy      # create tables
npm run db:seed                # load starter content from data/content.json
npm run dev                    # http://localhost:3000 (admin at /admin)
```

Useful commands: `npm run db:studio` (browse data), `npm run db:migrate`, `npm run db:push`, `npm run lint`, `npx tsc --noEmit` (typecheck).

No local Postgres? The site still renders — `lib/db.ts` falls back to seeded defaults via the circuit breaker; only admin saves require a reachable database.

10. Verification History (proven working)

- Typecheck and production build pass (14 routes).
- Local end-to-end with a real Postgres: login → save → row change confirmed directly in the database → public page reflects the change.
- Production: content API serves database data; admin save round-trip verified; Blob photo and CV uploads verified with publicly reachable URLs persisted to the DB; the image optimizer serves Blob images (HTTP 200, image/jpeg); clean domain alias verified.

11. Suggested Next Steps

1. **Database-backed password** — store the admin password as a bcrypt hash in Postgres so changes take effect instantly instead of after redeploy.
2. **Login rate limiting** — throttle the login endpoint to block brute-force attempts.
3. **Custom domain** — attach a purchased domain in Vercel project settings.
4. **Analytics** — Vercel Analytics or a lightweight page-view counter.
5. **Contact form** — serverless email delivery (e.g. Resend) instead of mailto links.